ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting

Anonymous Author(s)

Abstract

Design engineers lose the most time not writing fresh scripts but orchestrating design flows and patching the gaps where closed EDA tools refuse to interoperate—a flow that cannot run end-to-end is the top reason a design stalls. We argue that this is exactly where a general coding agent fits: it orchestrates flows natively (Makefile targets, Tcl drivers, scheduling scripts), and where tools leave gaps, its EDA-scripting ability synthesizes the missing glue—the kind of small, intent-to-code task at which LLMs excel. We present **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl that couples API retrieval, documentation inspection, and sandbox execution via unified MCP tools into a closed generate–execute–refine loop, with a *grounding-gap framework* that recasts EDA correctness as a *situated* property and proposes—as a testable claim rather than a proven bound—that the "grounding gap" is set by *observation fidelity*, rather than model scale. On **EDA-Scripting-Bench** (449 real-world tasks, three languages, four commercial runtimes), ZhuLong reaches **76.9%** pooled Pass@1 vs. **19.7%** (pure LLM) and **68.6%** (retrieval-augmented, no sandbox); the controlled read-back ladder (no read-back → binary → full observation, LLM fixed) spans 7.1 pp, and the system-level observation axis (pure LLM → full system) spans 57.2 pp—versus 23.7 pp for the backbone axis. ZhuLong is already deployed in production EDA flows at a leading memory design house. This reframes EDA scripting as the connective tissue that makes design flows runnable, not an end in itself.

CCS Concepts

• **Hardware** → **Software tools for EDA**; • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → *Software creation and management*.

Keywords

LLM Agents for EDA, EDA Scripting, Code Generation, Execution Grounding, Benchmarking

ACM Reference Format:

Anonymous Author(s). 2026. ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting. In *Proceedings of* . ACM, New York, NY, USA, 8 pages.

1 Introduction

Design engineers' daily work is less about writing fresh scripts than about orchestrating design flows and patching the gaps between EDA tools that refuse to interoperate—a flow that cannot run end-to-end is the single largest source of lost design time. Commercial EDA tools are closed: where they leave a gap, the only recourse is hand-written glue (a Makefile target, a Tcl driver, a SKILL wrapper), precisely the small intent-to-code synthesis at which LLMs excel. This reframes EDA scripting as the connective tissue that lets a general coding agent orchestrate flows natively and fill the breaks where tools stop. ZhuLong is built around this insight, and this paper asks which factor sets the ceiling of such an agent—grounded by a system deployed in production EDA flows at a leading memory design house.

We present **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl, built on Cline-CLI [4] and four unified MCP tools [13]—semantic and keyword API search, documentation lookup, and sandbox execution—that form a closed generate–execute–refine loop over four commercial runtimes via a single `lang–host` pair. An offline **API self-exploration** mechanism enriches the documentation prior as a bounded complement. Beyond the engineering, we give a **theoretical grounding** (Section 4): EDA correctness is a situated (L4) property, and a Bayesian argument bounds the grounding gap by observation fidelity, rather than model scale—a prediction the experiments could refute rather than a post-hoc explanation.

Contributions.

- **RQ1 (ceiling).** We formalize the grounding gap and measure it from both sides: the system-level axis (pure LLM, no observation → full system) spans 57.2 pp (LLM fixed), while the backbone axis spans 23.7 pp (harness fixed)—the system-level grounding range exceeds the backbone range (Section 4, 6).
- **RQ2 (necessary components).** Ablation isolates retrieval, execution, and belief update as jointly necessary, with toolchain-specific magnitudes that make them complementary rather than interchangeable; offline self-exploration is a bounded complement, not a substitute (Section 6).
- **RQ3 (toolchain generality). EDA-Scripting-Bench** (449 tasks, 4 runtimes, one schema) remeasures the same structure slice by slice; we specify the harness, schema, and assertion protocol in full (Section 5) so that the benchmark can be re-instantiated on other toolchains.

2 Background and Motivation

We target three widely-used EDA scripting interfaces across four commercial runtimes: PyAether for Empyrean Aether [7], SKILL for Cadence Virtuoso [16], and Tcl for Synopsys Custom WaveView and Cadence Innovus. Despite their prevalence, scripting for these environments poses challenges beyond what generic LLM code generators can address.

,

EDA scripting is fundamentally different. Prior LLM code generation succeeds where correctness is verified by unit tests [1, 3]. EDA scripting presents three distinct challenges: (i) APIs are long-tailed, tool-specific, and poorly documented; (ii) correctness depends on side effects in a live, stateful database, not return values; and (iii) the community lacks a standardized benchmark. The first two are technical; the third is infrastructural.

Prior custom/analog scripting work stops short of execution. LayoutCopilot [11] targets analog *layout* (not scripting) via multi-agent collaboration; ChaTCL [15] and RAG-EDA [14] retrieve from static knowledge bases to generate Tcl scripts but do not execute them, so runtime errors remain undetected and the gap between "looks plausible" and "actually works" stays unbridged. Execution feedback is not itself new—in the digital flow, ChatEDA [8], MCP4EDA [18], and AutoEDA [13] close the loop against synthesis and P&R state on open-source toolchains (Yosys, OpenLane, OpenRoad)—but none grounds against the live layout, schematic, or waveform database of custom/analog tools, the surface this paper targets. A recent survey [12] organizes EDA agents by handoff validity; ZhuLong targets the custom/analog scripting handoff, previously ungrounded. Concurrent benchmarks focus on digital Tcl [19] or physical-design agents [10]; our benchmark is the first for custom/analog scripting across three languages and four commercial runtimes.

Our approach. ZhuLong closes the loop from generation to execution to refinement, supplies a **theoretical grounding** (Section 4) explaining *why* execution is necessary, and provides the missing infrastructure: **EDA-Scripting-Bench** (Section 5).

3 System Design

3.1 Overall Architecture

ZhuLong is built on Cline-CLI [4], an open-source framework for autonomous coding agents, extended with EDA-specific retrieval, documentation, and execution capabilities. We call the resulting runtime an **ICL-type autonomous agent**: an agent that decides its own next action, keeps no explicit posterior, and maintains its belief state implicitly by in-context learning, as formalized in Section 4. The policy is kept commodity and unmodified, so any gain is attributable to the interface; the runtime's permission and hook mechanisms also supply the enforcement points on which the fixed evaluation protocol relies (Section 6.1). As shown in Figure 1, the system comprises three components: the LLM-based agent runtime, the API knowledge base, and the EDA execution environment, linked through four MCP tools, with an offline **API self-exploration** step pre-augmenting the knowledge base.

3.2 Four MCP Tools

The four tools jointly realize the observation channel Ω and the effector of Section 4, each accepting a unified `lang`–`host` pair that selects the target runtime so identical agent logic covers all four runtimes: `search_apis` retrieves relevant APIs by semantic similarity (BGE-M3 embeddings [2] with FAISS [6], top-K above a threshold, merged with BM25 keyword matches); `search_apis_by_keyword` retrieves by explicit keyword matching (CamelCase overlap plus BM25), complementing semantic search when a name fragment is known; `get_api_details` fetches full documentation (parameters, return values, examples), augmented by offline self-exploration when available; and `run_code` executes generated code in an isolated headless EDA sandbox and reads back a structured result (status, error log, stdout, return value) for iterative refinement, dispatching on the `lang`–`host` pair to resident or one-shot workers per runtime.

3.3 Agent Workflow

The agent iteratively refines its output through execution feedback [4]: reasoning over the instruction, retrieving and inspecting candidate APIs, generating and executing code, then re-planning based on the observed result. The agent is not constrained to a fixed sequence—it may skip retrieval when it already knows the API or conduct multiple rounds when initial results lack confidence—and terminates when it judges further attempts unlikely to succeed.

3.4 Offline API Self-Exploration (Complementary)

EDA API documentation is often incomplete, omitting parameter constraints, return structures, and error conditions. As Section 4 makes precise, retrieval-based augmentation enriches only the L1–L3 facts Ω can return, so it is complementary to—and bounded by—the observation of L4 that requires acting on the live design; we position self-exploration as an efficiency mechanism, not the primary accuracy driver. ZhuLong **proactively discovers** API behaviors through offline counterfactual experimentation in the sandbox: for each target API, the exploration agent reads available documentation, identifies gaps, generates targeted test scripts, executes them via `run_code`, and iteratively refines a behavioral model capturing parameter constraints, return structures, error conditions, and side effects. Exploration stops when a per-API confidence score exceeds threshold τ, when two consecutive rounds add no new constraint, or when a per-API execution budget M is exhausted; generated scripts are expert-gated to filter destructive operations. This offline process runs once per API and is served at runtime via `get_api_details` with no runtime overhead—removing it makes the agent slower (net-active time 645 vs. 464 s) and its token cost unchanged on average but far more variable (722 ± 116 vs. 710 ± 21 k; Table 3).

4 Theoretical Grounding: The Grounding Gap

This section develops the *grounding gap* framework, stated before the measurements of Section 6 so that its claims are

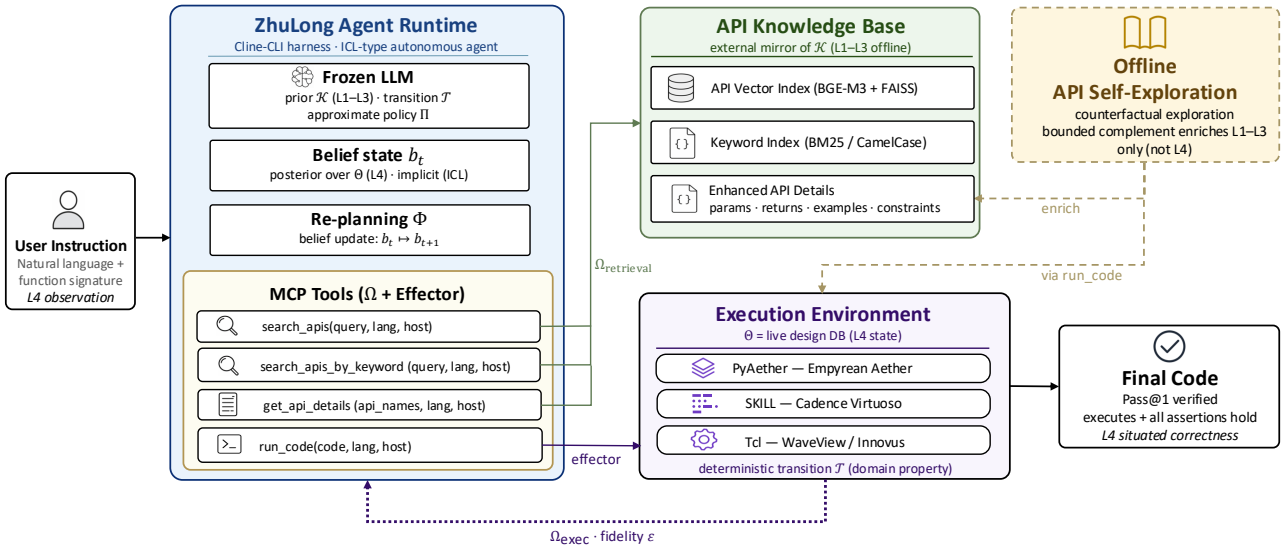


Figure 1: ZhuLong system architecture. The LLM agent runtime, the API knowledge base, and the EDA execution sandbox are linked through *four* unified MCP tools that dispatch on a single `lang`–`host` pair to *three* platforms (PyAether, SKILL, Tcl). Offline API self-exploration pre-augments the knowledge base (dashed); execution feedback is looped back for iterative refinement.

predictions the experiments could refute rather than explanations fitted afterwards: EDA correctness is a *situated* property that cannot be guaranteed from offline knowledge, execution is the only channel that observes the current design state, and the resulting gap is bounded by observation fidelity, rather than model scale.

4.1 A Four-Level Hierarchy of Information

To separate what can be learned offline from what must be observed online, Table 1 distinguishes four levels: L1 *syntactic correctness*, L2 *physical plausibility*, and L3 *historical factuality* (documented and past-oriented, frozen at the training cutoff) are learnable offline, whereas L4 *situated truthfulness*—what is true *right now* in a specific environment—is not, defined by indexicality (truth depends on context and changes over time), non-guaranteeability from offline data, and real-time commitment. L4 is what must be *read* from the environment rather than *recalled* from memory.

For EDA scripting, the situated state is the live design database—which cell and view are selected, and what the layout or schematic contains at this moment—and the user's current intent (so a clarification from the user is also an L4 observation). The correctness criterion we adopt—generated code executes without error and all assertions over the mutated design pass—is an L4 property: a script that is well-formed (L1), plausible (L2), and invokes documented functions (L3) can nonetheless violate an undocumented binding contract, depend on an internal selection state, or target the wrong cell. Such failures cannot be resolved offline; they must be *observed* by acting on the live design and reading back the result.

4.2 A Bayesian Formalization of the Grounding Gap

We cast grounding as decision-making under partial observability [9, 17]; the formulation is diagnostic rather than algorithmic—it makes precise what is missing (the observation likelihood Ω and the recursive update Φ) without prescribing tractable inference.

Let Θ denote the space of L4 states (the live EDA design database), o_t a runtime observation, a_t a generated code action, and $\mathcal{K}$ the offline knowledge (L1–L3) in the LLM's frozen parameters. The agent maintains a belief $b_t(\theta) \triangleq p(\theta_t \mid o_{1:t}, a_{1:t-1}, \mathcal{K})$ over the current L4 state, updated recursively by Bayes' rule:

$$b_t(\theta) \propto p(o_t \mid \theta_t) \int_{\Theta} p(\theta_t \mid \theta_{t-1}, a_{t-1})\, b_{t-1}(\theta_{t-1})\, d\theta_{t-1}, \quad (1)$$

where $p(o_t \mid \theta_t)$ is the *observation likelihood* Ω and $p(\theta_t \mid \theta_{t-1}, a_{t-1})$ is the *transition model* $\mathcal{T}$ supplied by the LLM's prior; actions are chosen by an approximate policy Π that minimizes expected cost under b_t, with $\mathcal{C}$ encoding the task assertions. Eq. (1) is the standard POMDP filter [9]; what is new here is not the filter but the *implementation* of Ω as retrieval plus execution read-back, which is this work's contribution to the POMDP stack.

Table 1: The four levels of information

Level	Name	Core Question	Learnable Offline?
L1	Syntactic Correctness	Is this string well-formed?	Yes
L2	Physical Plausibility	Could this event happen?	Yes
L3	Historical Factuality	Did this event happen?	Yes
L4	Situated Truthfulness	Is this true *right now*?	**No**

The *grounding gap* is the divergence between the posterior achievable with runtime observations and the posterior from the offline prior alone:

$$D_{\text{grounding}}(t) \triangleq D_{\text{KL}}(p(\cdot \mid o_{1:t}, \mathcal{K}, \text{env}) \parallel p_{\mathcal{K}}(\cdot)), \quad (2)$$

where the right-hand side is the *pure prior* unconditioned on any observation. We now bound this gap.

Scenario: the transition is deterministic. In EDA, `run_code` executes a script whose effect on the design database is deterministic: the same action in the same state yields the same result, so $\mathcal{T}$ is exact—a property of the domain, not a simplifying assumption. The only mismatch is in the observation likelihood Ω: retrieval may return incomplete documentation, and execution read-back may truncate—precisely what ϵ captures. Model mismatch under estimated parameters is a recognized obstacle [5]; our setting is milder, with only Ω, not $\mathcal{T}$, possibly mismatched.

Claim (observation-fidelity bound). Let $\epsilon \triangleq \sup_\theta \|p(\cdot \mid \theta) - \hat{p}(\cdot \mid \theta)\|_{\text{TV}}$, the total-variation distance between the true and the agent-used observation likelihoods. We posit that $D_{\text{grounding}} \leq g(\epsilon)$, with $g(\epsilon) \to 0$ as $\epsilon \to 0$ and *g independent of the complexity of the prior $\mathcal{K}$*: the grounding gap is bounded by observation fidelity, rather than model scale. We state this as a *prediction the experiments can test* rather than a theorem derived here; the next paragraph tests it from both sides.

Experimental correspondence. The read-back ladder ($\text{N}' \to \text{B} \to \text{F}$, Table 3) manipulates ϵ with retrieval and effector fixed; the monotone gain (7.1 pp, N' to F) reflects $g(\epsilon)$'s monotonicity, and the system-level ϵ axis (pure LLM to full system) spans 57.2 pp. The backbone axis (Table 4) changes $\mathcal{K}$ with ϵ fixed, and its 23.7 pp span is smaller—consistent with g being independent of $\mathcal{K}$. Section 6 develops both axes in full.

4.3 ZhuLong as an Instance of the Agent Harness

The formulation maps onto ZhuLong's architecture in three sentences. The frozen LLM supplies the prior $\mathcal{K}$ (L1–L3), the transition model $\mathcal{T}$, and the approximate policy Π; the Harness supplies the rest—the search tools are the observation modules Ω, `run_code` is the effector, and the re-planning loop (Section 3.3) implements Φ. The closed loop $b_t \to a_t \to o_{t+1} \to b_{t+1}$ turns one-shot generation into grounded search—a property of the coupled system, not the model alone—and ϵ is exactly the fidelity of *this* implementation of Ω (retrieval quality plus execution read-back), which is why the mapping must be retained.

4.4 Design Principles

The formalization yields three architectural principles. **Decouple knowledge from belief:** $\mathcal{K}$ is the static L1–L3 prior in the LLM's parameters, b_t the dynamic posterior the Harness maintains; adaptation is absorbed in b_t, not by updating $\mathcal{K}$ at deployment. **Optimize observation over the prior:** the gap is bounded by the fidelity of Ω ($g(\epsilon)$, independent of model scale), so investing in observation has a direct bounded effect while model scale gives diminishing returns once the prior is rich enough. **Design for continuous belief updating:** Φ runs online ($b_t \mapsto b_{t+1}$ per observation), so in the ICL-type agent of Section 3 what must be bounded is not a round count—there are none—but how much evidence the belief can accumulate: a graded budget on executions, or a read-back that lags the action it reports.

5 Benchmark Construction

5.1 Dataset: EDA-Scripting-Bench

EDA-Scripting-Bench is the instrument for both of this paper's comparisons: within one toolchain, the effect of each grounding component, and across toolchains, the effect of language structure. It covers PyAether, SKILL, and Tcl on four commercial runtimes, **449 tasks** in four slices (Table 2), all specified by one schema—a target language and runtime, a natural-language prompt, a function signature, and language-appropriate assertions—so a single harness (Section 3.3) scores all four platforms uniformly. Scoring is execution-based (a task passes only when the generated code acts correctly in the live runtime), and the cross-language slices are difficulty-controlled by construction (Section 5.3). Each task is a real break in a design flow where a closed tool stops and glue must be hand-written—a microcosm of the flow-orchestration gap this paper targets.

5.2 The PyAether slice (EDA-Eval-PyAether)

The reference slice, **EDA-Eval-PyAether** (158 tasks), anchors every task to a concrete PyAether API; EDA experts curated each assertion to be self-contained and executable in a clean sandbox without vendor-internal context. Tasks span layout, schematic, and design-management workflows, compose several distinct APIs, and sometimes depend on a

Table 2: Composition of EDA-Scripting-Bench

Slices and runtimes Slice	Runtime	Tasks
EDA-Eval-PyAether	Empyrean Aether	158
EDA-Eval-Skill	Cadence Virtuoso	157
EDA-Eval-WaveViewTcl	Synopsys Custom WaveView	77
EDA-Eval-InnovusTcl	Cadence Innovus	57
Total		**449**

technology library, so passing reflects genuine command of the tool's behavior rather than surface-level retrieval.

5.3 SKILL and Tcl slices

The remaining slices are built so cross-language scores are comparable. **EDA-Eval-Skill** (157 SKILL tasks on Cadence Virtuoso) is *function-aligned* with PyAether: each task re-implements the same entry point and functional specification as its PyAether counterpart, holding difficulty fixed. The **Tcl** slices are graded by difficulty across two further toolchains: waveform measurement on WaveView (**EDA-Eval-WaveViewTcl**, 77 tasks) and floorplanning, placement, and routing on Innovus (**EDA-Eval-InnovusTcl**, 57 tasks), so the cross-language comparison (Section 6.2.4) attributes score differences to language structure rather than task hardness.

5.4 Evaluation Protocol

A task passes if the generated code executes in its target runtime sandbox without errors and all assertions hold; the primary metric is Pass@1 [3], reported as mean ± s.d. over five independent runs (reshuffled task orderings, fresh agent contexts), per slice and pooled over all 449 tasks.

6 Experiments

6.1 Experimental Setup

Research questions. The framework of Section 4 makes three testable claims, one per RQ: **RQ1 (ceiling)**—is accuracy bounded by observation fidelity Ω or by the prior $\mathcal{K}$? (S1 varies Ω with the LLM fixed; S3 varies $\mathcal{K}$ with the harness fixed.) **RQ2 (necessary components)**—does bridging the gap require observation, execution, and continuous belief update, and can one substitute for another? (S2 varies the effector and the belief-update budget.) **RQ3 (toolchain generality)**—does the same structure reproduce on a different toolchain? (S4 re-runs S1–S2 on the SKILL and Tcl slices.)

Benchmark and protocol. We evaluate on **EDA-Scripting-Bench** (449 tasks, Section 5) using **Pass@1**. Each task runs in a single trace (the agent may iterate *within* it via Φ but is never restarted) with a 2500-second timeout; a file-based `PreToolUse` hook enforces a tool whitelist, blocks shell and code-search tools, and confines file access to a fresh working directory, so no assertions or reference solution is reachable—identical in every configuration, hence part of the fixed protocol. Each configuration is run five times (reshuffled order, fresh contexts), reported as mean ± s.d.; Δ is from table means. Task cases derive from non-redistributable vendor documentation (unlikely in public pretraining), and no third-party agent can execute these runtimes without the MCP bridge, so baselines are tool-registration ablations of the same fixed harness, making single-variable attribution possible.

Configurations. We isolate components by tool registration (a disabled tool is unregistered and invisible; the agent policy is held fixed), giving five configurations (Table 3): *Pure LLM* (no retrieval, sandbox, or self-exploration), *RAG* (retrieval + self-exploration, no sandbox; = w/o-Sandbox), *w/o Self-Expl.* (retrieval + sandbox), *w/o Retrieval* (sandbox + self-exploration), and *ZhuLong (Full)* (all three on). S1 probes the observation axes at their extremes; S2 bounds Φ. All main experiments use DeepSeek-V4-Pro as the backbone; S3 evaluates five additional backbones under the identical harness.

6.2 Results and Ablations

Table 3 gives the five configurations pooled and per slice. Relative to the full system, removing execution costs 8.3 pp pooled, removing retrieval 10.7 pp, and removing self-exploration 2.4 pp; against the pure-LLM floor the full system gains 57.2 pp. Wilson 95% intervals on the pooled Pass@1 (n=2245): Full [75.11, 78.60], w/o-Self-Expl. [72.66, 76.26], w/o-Retrieval [64.22, 68.13], RAG [66.65, 70.49], N′ [67.87, 71.66], B [70.31, 74.01], Pure-LLM [18.11, 21.40]; the controlled $N' \to F$ read-back gap of 7.1 pp has 95% interval [4.52, 9.68] pp (SE= 1.32 pp). A paired bootstrap is deferred: batch directories store only counts, so per-task arrays would require a one-shot regrade over both arms' batches. Where each component's contribution comes from is toolchain-specific (Section 6.2.4).

6.2.1 S1: Observation fidelity (RQ1). Principle 2 predicts that with the model fixed, accuracy is bounded by the fidelity of Ω. **Documentation axis.** Holding the agent loop, effector, and LLM fixed, removing retrieval costs 10.7 pp pooled, but highly toolchain-specific: on PyAether the read-back channel partially substitutes for retrieval (86.7% of traces introspect via the sandbox using `dir()`/`inspect`/`__doc__`, reconstructing the API knowledge retrieval would return), whereas on SKILL and Tcl the API surfaces cannot be recovered from execution alone, so removing retrieval removes the knowledge. Retrieval is thus one *implementation* of Ω, not the channel itself.

Read-back axis. The second axis is how much of the live state execution returns. We compare three closed-loop variants differing only here (Table 3, lower block): (N′) *no read-back*—`run_code` executes but returns nothing; (B) *binary feedback*—pass/fail only; (F) *full observation*—stdout, traces, per-assertion outcomes, and queried state (the default). The predicted monotone ordering N′ < B < F holds pooled and on the two largest slices; on Tcl it inverts at the

Table 3: Main results on EDA-Scripting-Bench: Pass@1 (%) over five runs (mean ± s.d.), pooled over all 449 tasks and per slice. The three right-most columns report per-task operating cost over all 449 tasks (five runs, mean ± s.d.). The lower block varies the *read-back* Ω alone (retrieval and effector fixed); (F) coincides with the Full row.

Configuration	Overall (449)	PyAether (158)	SKILL (157)	Tcl (134)	Time (s)	run_code (per task, 449)	Input (k)
Baseline 1: Pure LLM	19.7 ± 1.4	14.3 ± 2.1	33.2 ± 2.3	10.1 ± 1.1	183 ± 9	0	72 ± 2
Baseline 2: RAG (retr.+self-expl., no sandbox)	68.6 ± 1.6	71.1 ± 4.6	52.2 ± 1.9	84.6 ± 1.9	226 ± 9	0	272 ± 8
ZhuLong w/o Retrieval (execution only)	66.2 ± 4.2	73.4 ± 7.3	45.2 ± 9.4	82.2 ± 2.3	981 ± 151	22.6 ± 2.8	658 ± 133
ZhuLong w/o Self-Expl.	74.5 ± 1.8	77.0 ± 4.1	**61.5 ± 0.7**	86.7 ± 3.0	645 ± 132	12.6 ± 1.4	722 ± 116
ZhuLong (Full)	**76.9 ± 2.3**	**82.5 ± 2.0**	60.3 ± 6.7	**89.9 ± 2.0**	464 ± 57	10.6 ± 0.7	710 ± 21
Read-back axis (retrieval + effector fixed, only what run_code *returns varies)*							
(N′) No read-back (prior only)	69.8 ± 1.9	73.3 ± 2.2	50.7 ± 4.9	87.9 ± 1.7	—	—	—
(B) Binary pass/fail	72.2 ± 1.2	78.1 ± 1.3	57.2 ± 4.8	83.0 ± 2.3	—	—	—
(F) Full observation	**76.9 ± 2.3**	**82.5 ± 2.0**	**60.3 ± 6.7**	**89.9 ± 2.0**	464 ± 57	10.6 ± 0.7	710 ± 21

Bold marks the per-column best. The Tcl column pools the two Tcl hosts, WaveView (77 tasks) and Innovus (57), by summing pass counts per run over 134 tasks. Time = net-active solve time (final-session active-seconds, excluding quota wait); run_code = calls per task; Input = input tokens per task (thousands), each averaged over all 449 tasks (five runs). Read-back-axis rows share the Full cost (F) or are not separately costed (N′, B).

middle rung, because Tcl scripts are short and near-single-shot, largely solved from documentation, so a binary signal supplies no actionable information. This is the ϵ-axis the claim of Section 4.2 predicts tracks accuracy; the N′-to-F gap operationalizes the grounding gap as an accuracy difference.

Self-exploration rung. The top rung adds constraints discovered offline (Section 3.4); per the framework this enriches only the L1–L3 facts Ω can return, so it should move the result less than the channel's own fidelity. Removing it (retrieval and effector kept) drops Pass@1 from 82.5% to 77.0% on PyAether (76.9% to 74.5% pooled), a bounded prior-side complement rather than a substitute. Self-exploration is the one cost-negative rung (cutting net-active solve time by 28.1%, see Table 3), whereas removing retrieval makes the agent fall back on blind sandbox probing (2.1× the full system's time).

6.2.2 S2: The closed loop (RQ2). Principle 3 requires that actions reach the design through a causal interface and that Φ run continuously. **Effector.** Removing the sandbox drops pooled Pass@1 from 76.9% to 68.6% (−8.3 pp): without an effector, b_t receives no live-state evidence and the agent is a prior-only predictor. **Belief update.** We bound how often the agent *may* execute (the number of `run_code` calls per trace, denied at the tool boundary once spent) rather than forcing early submission, and vary the evidence along two axes: the budget k, and a *lagged* channel that returns the *previous* execution's state, leaving the agent one action behind the design. On PyAether, accuracy rises monotonically with the budget (63.8% at k=1, 69.1% at k=3, 77.7% at k=10, unsaturated); the lagged channel lands between k=3 and k=10 (73.0%)—stale evidence is worth markedly less than fresh—and tighter budgets leave traces unfinished (Converged drops from 99.1 to 76.3 at k=1).

6.2.3 S3: The prior axis (the prior-side contrast of RQ1). Principle 1 separates knowledge from belief: the frozen prior should bound what a harness can achieve without substituting for observation. Table 4 changes only $\mathcal{K}$ (the LLM) with the harness fixed; DeepSeek-V4-Pro is the primary backbone,

Table 4: Impact of LLM backbone (ZhuLong, full harness).

Backbone	PyAether (158)	SKILL (157)	Tcl (134)	Overall (449)	Δ
DeepSeek-V4-Pro	82.5±2.0	**60.3±6.7**	**89.9±2.0**	**76.9±2.3**	—
GLM-5.3-Flash	**86.8±1.6**	57.2±6.9	80.3±3.8	74.5±2.4	−2.4
GLM-5.2	83.3±3.1	54.4±4.5	82.7±1.6	73.0±2.1	−3.9
Kimi-K2.6	77.0±1.6	55.5±2.7	75.8±1.9	69.1±0.5	−7.8
DeepSeek-V4-Flash	60.1±5.5	47.0±1.8	70.4±3.1	58.6±2.3	−18.3
Doubao-Seed-2.0-Pro	63.8±1.4	42.5±3.4	53.3±3.4	53.2±1.9	−23.7

Pass@1 (%), five runs (mean ± s.d.); Tcl pools WaveView (77) and Innovus (57), by summing pass counts per run over 134 tasks; Δ vs. the primary backbone, pooled. Bold = per-column best. The strongest-to-weakest span is 76.9 − 53.2 = 23.7 pp. Per-host WaveView and Innovus Pass@1 for all configurations are reported in the supplementary material.

yet the backbone remains decisive—even with retrieval, sandbox, and self-exploration in place, the strongest-to-weakest spread is 23.7 pp pooled. Read against Principle 2, this spread is what the $\mathcal{K}$ axis can move, and it does not substitute for observation: the system-level observation-fidelity axis moves the result by more than the entire backbone range (57.2 vs. 23.7 pp pooled), exactly the prediction of the claim in Section 4.2. GLM-5.3-Flash beats the primary on PyAether yet loses 2.4 pp pooled—slice-level snapshots can invert backbone rankings, itself an argument for the pooled protocol.

6.2.4 S4: The toolchain axis (RQ3). The four slices share one schema, assertion criterion, and harness, so S1–S2 re-runs slice by slice with nothing changed but the toolchain. Table 3 carries the comparison: the pure-LLM floor and the advantage of closed-loop configurations reproduce on every slice (the one exception being SKILL, where the full system trails its w/o-Self-Exploration variant by 1.2 pp), and the read-back ordering N′ < B < F holds on the two largest slices and pooled (the Tcl inversion is disclosed in Section 6.2.1). Component magnitudes are toolchain-specific—execution buys +8.1 pp on SKILL but only +5.3 on Tcl, and retrieval buys +15.1 pp on SKILL, +9.1 pp on PyAether, and +7.7 pp on Tcl—but the structure (observation and execution jointly closing the loop) reproduces on every slice.

6.3 Error Analysis

On PyAether, ZhuLong reaches 82.5% Pass@1 (130/158). Generation-stage failures are a harness-budget, not a grounding, phenomenon—mostly timeouts on multi-step tasks or message-budget exhaustion cycling through `search_apis`. Manual inspection of execution-stage failures yields two root causes: 12 **API-misuse** cases (correct API family, but an undocumented C++-binding contract—hidden types, internal state, or context preconditions—is violated), and 9 **API-composition errors** (valid calls composed incorrectly, or a mandated call replaced by a plausible approximation, e.g., a substring match for equality). Both share one signature: the violated constraint is invisible in documentation and only weakly exposed by execution—the L4 invisibility Section 4 predicts situated correctness hinges on.

7 Discussion

Section 4 elevates the error analysis of Section 6.3 from observation to structural principle: in domains with long-tailed, under-documented APIs, the grounding gap is bounded by observation fidelity, so execution channels—not larger priors—are binding, a paradigm that may generalize to other domain-specific scripting tasks.

Three limitations: base-LLM performance varies significantly (53.2%–76.9% pooled) and the benchmark size limits power to certify small effects, so the self-exploration gain (+2.4 pp) is complementary, not headline. Offline self-exploration incurs a one-time upfront cost per API (amortized), and the interactive bridge is single-threaded, limiting concurrency. Per-slice dispersions can be large for ablated configurations (e.g., SKILL w/o Retrieval ±9.4), so slice-level comparisons need care; Pass@1 is assertion-based, certifying the code produces the effects the assertions check, not full functional equivalence. The SKILL self-exploration regression (−1.2 pp) is consistent with SKILL's smaller API surface: offline self-exploration enriches L1–L3 facts, but on SKILL the marginal fact is more often redundant with what retrieval already returns, crowding out task-relevant evidence. Deployment metrics (adoption, task completion, engineer time saved) are subject to a separate disclosure review and will be reported in an extended version.

8 Conclusion and Future Work

ZhuLong reframes EDA scripting as the connective tissue of design-flow orchestration: a general coding agent orchestrates flows natively, and where closed tools leave gaps, its EDA-scripting ability synthesizes the missing glue. The execution-grounded loop and the grounding-gap bound (observation fidelity, rather than model scale; Section 4) explain why this works: EDA correctness is situated, so the agent must read the environment's API facts and read back its code's effect on the live design. The complete system reaches 76.9% pooled Pass@1 and is deployed in production EDA flows at a leading memory design house—making design flows runnable end-to-end, not writing more scripts.

Four directions invite future work: (i) *domain-specializing the harness* (retrieval and context compression tailored to EDA's long-tailed, stateful APIs); (ii) *long-term memory and self-evolution* (accumulating verified API constraints across runs); (iii) *multimodal grounding* (schematics, waveforms, and layout images extending Ω); and (iv) *benchmarking design-flow orchestration* (end-to-end flow metrics, so the connective-tissue claim can be measured rather than only asserted).

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732

[2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs.CL] https://arxiv.org/abs/2402.03216

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374

[4] Cline Contributors. 2024. Cline: Autonomous AI Coding Assistant. https://github.com/cline/cline.

[5] Erick Delage and Shie Mannor. 2010. Percentile Optimization for Markov Decision Processes with Parameter Uncertainty. *Operations Research* 58, 1 (2010), 203–213.

[6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. arXiv:2401.08281 [cs.LG]

[7] Empyrean Technology. 2026. Empyrean PyAether: Full Custom Design Platform Ecosystem. https://empyrean.com.cn/solutions/index-128.html. Accessed: 2026-06-28.

[8] Zhuolun He, Haoyuan Wu, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. arXiv:2308.10204

[9] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* 101, 1-2 (1998), 99–134.

[10] Qiufeng Li, Rongqian Chen, Quan Cheng, Chengxuan Wang, Sizhe Tang, Wuxi Li, Duo Ding, Chia-Tung Ho, Haoxing Ren, David Z. Pan, Tian Lan, and Weidong Cao. 2026. PDAGENT-BENCH: Characterizing, Grounding, and Architecting LLM Agents for VLSI Physical Design. *arXiv preprint arXiv:2606.17253* (2026).

[11] Bingyang Liu, Haoyi Zhang, Xiaohan Gao, Zichen Kong, Xiyuan Tang, Yibo Lin, Runsheng Wang, and Ru Huang. 2025. LayoutCopilot: An LLM-Powered Multiagent Collaborative Framework for Interactive Analog Layout Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 44, 8 (2025), 3126–3139. doi:10.1109/TCAD.2025.3529805

[12] Jiawei Liu, Peiyi Han, Yuntao Lu, Su Zheng, Fengyu Yan, and Bei Yu. 2026. Agentic Electronic Design Automation: A Handoff Perspective. arXiv:2606.19795

[13] Yiyi Lu, Hoi Ian Au, Junyao Zhang, Jingyu Pan, Yiting Wang, Ang Li, Jianyi Zhang, and Yiran Chen. 2025. AutoEDA: Enabling EDA Flow Automation through Microservice-Based LLM Agents. Uses MCP protocol for EDA automation. arXiv:2508.01012 https://arxiv.org/abs/2508.01012

[14] Yuan Pu, Zhuolun He, Tairu Qiu, Haoyuan Wu, and Bei Yu. 2025. Customized Retrieval Augmented Generation and Benchmarking for EDA Tool Documentation QA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 44 (2025). arXiv:2407.15353

[15] Yibo Rui, Yuanhang Li, Rui Wang, Ruiqi Chen, Yanxiang Zhu, Zhixiong Di, Xi Wang, and Ming Ling. 2025. ChaTCL: LLM-Based Multi-Agent RAG Framework for TCL Script Generation. In *Proceedings of the International Symposium of Electronics Design Automation (ISEDA)*. IEEE, New York, NY, USA, 736–742.

[16] Cadence Design Systems. 2024. *Virtuoso Studio SKILL Language Reference*. https://www.cadence.com

[17] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. 2005. *Probabilistic Robotics*. MIT Press, Cambridge, MA, USA.

[18] Yiting Wang, Wanghao Ye, Yexiao He, Yiran Chen, Gang Qu, and Ang Li. 2025. MCP4EDA: LLM-Powered Model Context Protocol RTL-to-GDSII Automation with Backend Aware Synthesis Optimization. arXiv:2507.19570

[19] Ning Xu, Zhaoyang Zhang, Senlin Shu, Qi Lei, Jiaqi Lv, Wensuo Wang, Tianhao Zhao, Chao Zhang, Zhaoliang Yang, Xiangyu Li, Zhaorui Su, Jingshan Li, and Xin Geng. 2026. iScript: A Domain-Adapted Large Language Model and Benchmark for Physical Design Tcl Script Generation. *arXiv preprint arXiv:2603.04476* (2026).